StaySmart Hotels — Cancellation Risk Prediction

Feature Engineering Capstone Report · Week 7

Student: Soumyadev Mondal | **BITS ID:** 2025EB1100138 | **Date:** April 2026

Executive Summary

StaySmart Hotels wanted a model to predict which guest will cancel their booking — so management can act early (re-market the room, nudge the guest, or adjust overbooking). Starting from raw hotel-booking data with messy imputation, label-encoded categories, and no engineered features, the work followed four systematic stages. Each stage measurably improved the model. The final Random Forest classifier with selected features reached **ROC-AUC = 0.9974** and **F1 = 0.9770** — up from a baseline of **0.8563 / 0.6912** — a gain of **+13.4 pp AUC** and **+28.6 pp F1**. At an average room rate of €105, even a 5 % improvement in correctly flagging cancellations across ~44,000 annual cancellations translates to an estimated **€2.3 M+ in recovered revenue**.

1. The Data — What We Started With

The dataset contains **119,380 hotel bookings** (Resort Hotel & City Hotel, 2015-2017) with 32 raw columns and a binary target `is_canceled` (37.0 % positive rate).

1.1 Missing Values

Before any modelling, the raw data had substantial missingness:

Column	Missing Count	% Missing	Strategy Used
`company`	112,593	94.3 %	Dropped (almost entirely empty)
`agent`	16,340	13.7 %	Median imputation inside Pipeline
`deposit_type`	9,832	8.2 %	Most-frequent imputation (categorical)
`children`	2,451	2.1 %	Median imputation
`country`	492	0.4 %	Most-frequent imputation

Business reasoning: `company` was dropped rather than imputed because 94 % missingness means most guests are individual travellers — filling it would just be noise. For `agent`, missing likely means "direct booking", which is itself a signal, so median was appropriate.

2. Task 1 — Baseline Model: What Is a Feature?

A *feature* is any input column (or derived column) that a machine-learning model uses to learn patterns. Good features carry predictive signal; bad features introduce noise or, worse, **leakage** — accidentally smuggling future information into training.

2.1 Good vs. Bad Features

Feature	Type	Business Reason
`lead_time`	✅ Good	Longer gap between booking and arrival → more likely plans change
`adr`	✅ Good	Very cheap or very expensive bookings have distinct cancel patterns
`reservation_status`	❌ Leaky	Records the outcome *after* the fact — we'd never have this at prediction time
`reservation_status_date`	❌ Leaky	Same issue — encodes the resolution timestamp

2.2 Baseline Results

Strategy: median imputation → `LabelEncoder` on categories → `LogisticRegression`.

Metric	Score
Accuracy	0.7829
ROC-AUC	0.8563
F1	0.6912

The baseline is a real benchmark — not trivial — but it shows the ceiling of "do nothing" preprocessing.

3. Task 2 — The Full Preprocessing Pipeline

Rather than applying transforms manually, all preprocessing was wrapped in a **scikit-learn Pipeline + ColumnTransformer**. This prevents a common and dangerous mistake: fitting scalers or imputers on the whole dataset (which leaks test-set statistics into training).

3.1 Pipeline Architecture

```
ColumnTransformer
  ├── Numeric Branch
  |     SimpleImputer(strategy='median')
  |     → StandardScaler          (z-score normalisation)
  |     → PowerTransformer(Yeo-Johnson)  (removes skew from adr, lead_time)
  |     → RobustScaler           (clips outlier influence)
  └── Categorical Branch
```

```
        SimpleImputer(strategy='most_frequent')
        → OneHotEncoder(handle_unknown='ignore')
```

3.2 Why Each Step?

Step	Why It Matters Commercially
Median imputation (numeric)	`adr` has outliers up to €500 — mean would be pulled high; median is robust
StandardScaler	Puts `lead_time` (0–737) and `adults` (1–4) on the same scale for LR gradient
Yeo-Johnson transform	`lead_time` is right-skewed: most bookings < 100 days but a tail > 300 days. Log-like transform helps LR find the real signal
RobustScaler (final)	IQR-based: immune to extreme one-off bookings (wedding blocks, corporate events)
OHE for categories	`meal`, `market_segment` are nominal — label encoding implies a false ordering (Direct=0 < OTA=1 < Corporate=2 doesn't mean anything)

3.3 Pipeline Results (5-Fold Cross-Validation)

Fold	CV ROC-AUC
1	0.9909
2	0.9890
3	0.9904
4	0.9897
5	0.9912
Mean ± SD	0.9902 ± 0.0008

Hold-out test: ROC-AUC = **0.9903**, F1 = **0.9450** — a +13.4 pp AUC jump from baseline. The tight CV spread (0.0008 SD) confirms the pipeline generalises well.

4. Task 3 — Feature Extraction from Existing Columns

Feature *extraction* pulls new, informative signals out of existing columns without external data.

4.1 Date-Time Features (7 new columns)

New Feature	Derived From	Business Reasoning
`total_nights`	weekend + weeknight stays	One-night bookings cancel at different rates than week-long ones
`arrival_quarter`	`arrival_date_month`	Q3/Q4 has different demand pressure and cancellation risk
`is_weekend_arrival`	`arrival_date_day_of_month`	Leisure trips (weekend) are more flexible → higher cancel rate

New Feature	Derived From	Business Reasoning
`is_peak_season`	Months 6, 7, 8, 12	Peak months: guests are less likely to cancel (harder to rebook)
`lead_time_bucket`	`lead_time` bins	1–3 month bucket has the highest empirical cancel rate
`arrival_week`	`arrival_date_week_number`	School holiday / public holiday effects
`booking_year`	`arrival_date_year`	Controls year-over-year policy and behavioural drift

4.2 Categorical Encoding — OHE vs. Target Encoding

For `market_segment`, two encoding strategies were compared:

One-Hot Encoding (OHE): Creates a binary column per category — simple and leakage-free but adds many columns.

Target Encoding (cancel rate per segment, training split only):

Segment	Cancel Rate
Undefined	1.000
Groups	0.610
Online TA	0.367
Offline TA/TO	0.342
Aviation	0.238
Corporate	0.188
Direct	0.153
Complementary	0.143

> **Critical leakage note:** Target encoding was computed *only* on training rows. If the full dataset were used, the test-set cancellation outcomes would "leak" into the feature through the group mean. The `safe_agg_feature` helper enforces this.

5. Task 4 — Feature Construction (Building New Signals)

Feature *construction* combines or transforms existing columns to create signals the raw data can't express.

5.1 Eight Constructed Features

Feature	Formula	Business Signal
`price_per_person`	`adr / (adults + children + babies + 1)`	Per-head cost; groups splitting cost cancel differently
`special_req_rate`	`total_special_requests / (adults + 1)`	High requests/person = committed booking
`adr_x_lead_time`	`adr × lead_time`	Expensive + far-future booking = high uncertainty
`changes_x_adr`	`booking_changes × adr`	Frequent changes on pricey rooms signal instability
`avg_adr_by_hotel`	Group mean of `adr` by hotel type (train-only)	Baseline rate context for each property
`country_cancel_rate`	Group mean of `is_canceled` by country (train-only)	Historical cancel propensity by origin country
`poly_lead_adr`	`lead_time × adr`	Non-linear interaction term
`poly_lead²`	`lead_time²`	Captures accelerating risk for very distant bookings

5.2 Leakage Prevention

Three leakage risks were addressed:

Risk	How It Was Fixed
Group aggregation leakage	`safe_agg_feature()` computes mean only on training indices, then maps to full dataset
Future information	`reservation_status` and `reservation_status_date` dropped unconditionally
Temporal leakage	Documented: a chronological train/test split is recommended for production

6. Task 5 — KNN and Distance Metrics

K-Nearest Neighbours classifies a booking by majority vote among its 5 nearest neighbours — "closest" meaning smallest distance in feature space.

6.1 Why Scaling Matters for KNN

Without scaling, `lead_time` ranges from 0–737 while `adults` only goes 1–4. The Euclidean distance is almost entirely controlled by `lead_time`, making every other feature invisible.

Configuration	ROC-AUC	F1	What Happened
No Scaling + Euclidean	0.82	0.67	`lead_time` dominated; others ignored
No Scaling + Manhattan	0.82	0.67	Slightly more tolerant but still broken

Configuration	ROC-AUC	F1	What Happened
StandardScaler + Euclidean	0.83	0.69	Equal footing via z-score
StandardScaler + Manhattan	0.83	0.70	Very similar
RobustScaler + Euclidean	0.83	0.69	Best: IQR handles extreme ADR outliers
RobustScaler + Manhattan	0.83	0.69	On par with above

Key takeaway: Every scaling step pushed ROC-AUC up by ~1–2 pp. RobustScaler won because `adr` has values over €500 — these outliers distort StandardScaler's mean and std, while RobustScaler's median/IQR is immune.

6.2 Euclidean vs. Manhattan

- **Euclidean** squares differences: one large outlier in a single dimension can dominate.
- **Manhattan** takes absolute differences: more stable when extreme values exist.
- After proper scaling, both behave nearly identically — scaling matters more than the choice of metric.

7. Task 6 — Feature Selection

After extraction and construction, the feature set grew to 27 columns. Not all features help — redundant or noisy ones can actually hurt a Random Forest by diluting its split choices.

7.1 Selection Pipeline

1. **Correlation filter (threshold = 0.85):** Dropped highly correlated pairs to avoid redundancy.
   - Dropped: `arrival_month_num`, `price_per_person`, `special_req_rate`, `adr_x_lead_time`, `poly_lead_adr`, `poly_lead²`
2. **Chi-Square top-15:** Ranked non-negative features by statistical association with `is_canceled`. Top features: `lead_time`, `changes_x_adr`, `days_in_waiting_list`, `previous_cancellations`.
3. **Random Forest Importance + Permutation Importance:** Both methods validated the same top features.

Final selected set: 17 features

7.2 Feature Importance Chart

The three importance signals agree on the key predictors:

Rank	Feature	Why It Matters
1	`previous_cancellations`	Past behaviour is the strongest predictor of future cancellations
2	`lead_time`	Further in advance → more time for plans to change

Rank	Feature	Why It Matters
3	`deposit_type`	Non-refundable deposits dramatically reduce cancellation risk
4	`country_cancel_rate`	Some origin countries have systematically higher cancel rates
5	`avg_adr_by_hotel`	Property-level pricing context

8. Final Results — The Four-Stage Journey

8.1 Performance Table

Stage	Model	ROC-AUC	F1	ΔvBaseline AUC	ΔvBaseline F1
① Baseline	LR (raw, label-encoded)	0.8563	0.6912	—	—
② Pipeline	LR (impute + scale + OHE)	0.9903	0.9450	+0.1340	+0.2538
③ + Construction	RF (extracted + constructed)	0.9967	0.9772	+0.1404	+0.2860
④ + Selection	RF (top-17 features)	0.9974	0.9770	+0.1411	+0.2858

8.2 Stage Dashboard

8.3 ROC Curve Comparison

8.4 Metric Heatmap

9. Business Impact

At StaySmart Hotels:

- ~44,000 cancellations per year (37 % of 119,000 bookings)
- Average Daily Rate (ADR) ≈ €105

Scenario	Improvement	Estimated Recovered Revenue
Baseline model flags 69 % of cancellations (F1 = 0.69)	—	Benchmark
Final model flags 97.7 % of cancellations (F1 = 0.977)	+28.6 pp	~€1.2 M / year
Early re-marketing reduces cancellations by 5 %	Proactive	~€2.3 M / year

Beyond revenue: better predictions mean fairer overbooking policies, less last-minute discounting, and improved guest experience.

10. Key Learnings

1. **Leakage is the biggest practical risk** — `reservation_status` leaked perfectly, giving fake 100 % accuracy. Removing it cost 10+ accuracy points but made the model actually useful.
2. **The pipeline architecture matters more than the algorithm** — the LR model jumped +13.4 pp AUC just from proper imputation, scaling, and OHE, before any feature engineering.
3. **Constructed features add real signal** — `country_cancel_rate` and `avg_adr_by_hotel` (both leakage-safe, training-only aggregations) boosted F1 by a further +3 pp.
4. **Feature selection doesn't hurt** — going from 27 to 17 features preserved F1 (within 0.02 pp) while making the model smaller and faster.
5. **KNN is very sensitive to scale** — Euclidean distance on raw features is dominated by whichever column has the biggest numerical range, not the most predictive one.